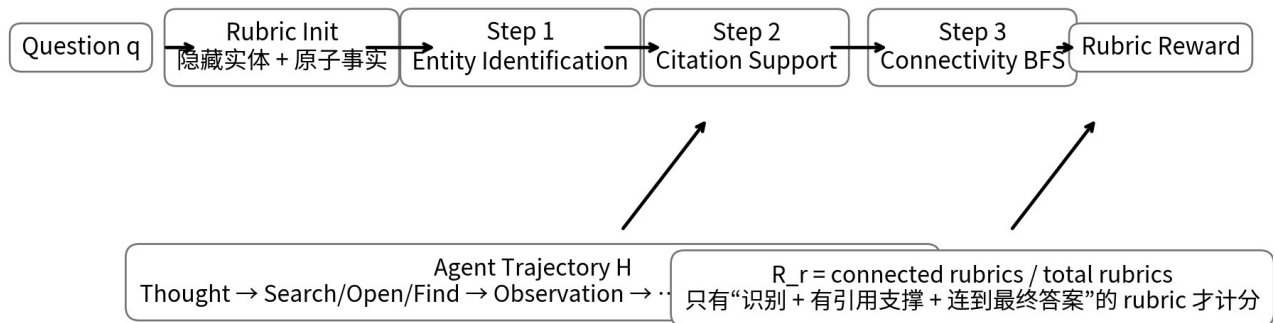


CaRR

Citation-Aware Rubric Rewards for Robust Deep Search RL

完整技术研究 · Know-Why × Know-How × Reward / Evidence / RL / Runtime



核心机制：从正确答案奖励，升级到“实体识别 - 引用支撑 - 证据连通”的可验证奖励链

论文：arXiv:2601.06021 · ACL 2026（官方仓库公告）

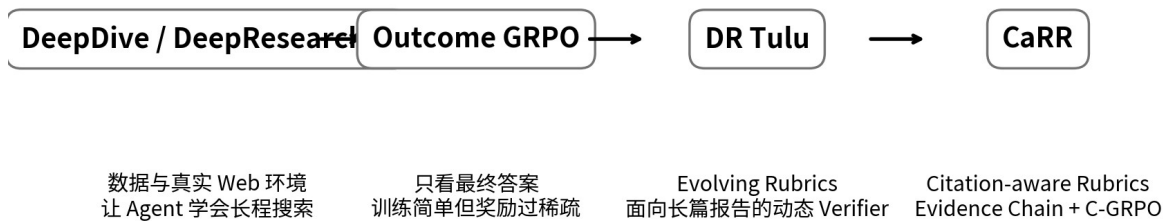
研究快照：2026-08-13

阅读导航与核心判断

术语校正 CaRR 在论文中指 Citation-Aware Rubric Rewards，不是一个“Corrective Agent”框架。它的核心对象是 Deep Search RL 的 Reward / Verifier。

一句话结论 CaRR 发现：Deep Search Agent 仅仅“最终答对”并不足以证明它学会了研究；真正稳健的 Research Policy 应该能显式识别中间实体、用实际网页证据支撑事实，并把这些证据连接成通往最终答案的完整 evidence chain。

如果把 DeepResearcher / DeepDive 的核心贡献理解为“如何构造可训练的搜索环境与数据”，把 DR Tulu 理解为“如何奖励长篇研究报告”，那么 CaRR 解决的是一个更窄但极其关键的问题：如何防止 outcome-based RL 把 shortcut、幸运幻觉和不完整验证当成正确策略继续强化。[S1][S2]



Reward 演进：Answer Correct → Process Coverage → Citation Grounding → Evidence Connectivity

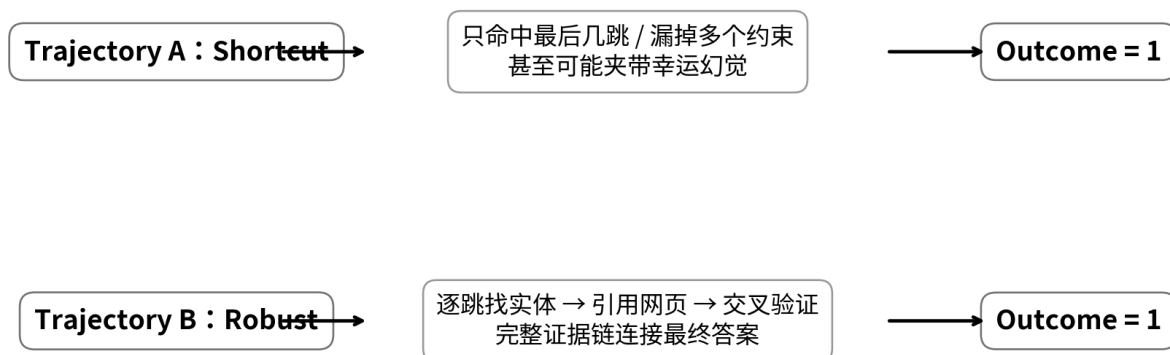
图 1 | CaRR 在 Deep Research RL 演进中的位置

最值得带走的 8 个结论

- Outcome reward 是“结果验证器”，不是“研究质量验证器”。同样答对，轨迹质量可以完全不同。
- Fine-grained reward 不能只看是否出现中间实体；必须进一步验证事实是否被真实引用支撑。
- Citation 也不够：孤立的正确事实可能与最终答案无关，因此必须检查 evidence connectivity。
- CaRR 最聪明的设计之一，是把 LLM 擅长的语义判断与 BFS 擅长的结构连通性判断组合起来。
- C-GRPO 只给“最终答对”的 rollout 加 rubric bonus；论文消融显示给所有 rollout 加 bonus 会显著伤害训练。
- 稳健 Search Policy 的表现不仅看固定预算 accuracy，还应看 test-time scaling：给更多 context/tool budget 后是否继续受益。
- 引用与 provenance 不应只在报告生成末尾补格式，而可以直接成为训练信号的一部分。
- CaRR 是 Reward/Verifier Layer，不是完整 Deep Research runtime；最适合与 DeepDive/Tongyi/自研 Agent runtime 组合。

1. 为什么 CaRR 出现：Outcome-only RL 的结构性缺陷

Deep Search RL 常用合成 multi-hop QA：答案是一个短实体字符串，因此 reward 可以非常便宜地做成 0/1——最终答案等于 ground truth 就给 1，否则给 0。这种设置极适合大规模 GRPO，但它把“搜索与推理过程是否可靠”全部压扁成一个 bit。论文明确指出两类失败模式：shortcut exploitation 与 hallucination tolerance。[S1]



Outcome-only RL 无法区分两条轨迹，因而会把“碰巧答对”的坏策略也强化

图 2 | 为什么 Answer Correct 不能代表 Research Correct

1.1 Shortcut exploitation 为什么会被强化

合成问题往往包含多跳约束。模型可能只利用问题末尾的少数线索直接猜到答案，而没有验证前面的约束；只要最终字符串正确，outcome reward 就无法区分这种“少做研究”的策略。随着 RL 进行，模型会发现更短、更省 tool call 的路径同样能拿满分，于是形成局部最优。论文训练动态正观察到：GRPO 的 tool-call steps 在训练后期持续下降，而 C-GRPO 在初始效率提升后又逐步增加证据搜集。[S1]

1.2 幸运幻觉为什么危险

Agent 可能生成一个正确实体，却没有真实证据支撑，或者把网页内容错误归因到某个实体。纯 outcome reward 仍会把这条轨迹当成正样本。CaRR 的思路是把“答对”拆成三个更细的问题：你是否明确找到了关键实体？事实是否被你实际引用的网页支持？这些事实是否真的组成连接到最终答案的证据链？

2. CaRR 方法：把多跳问题变成可验证 Rubrics

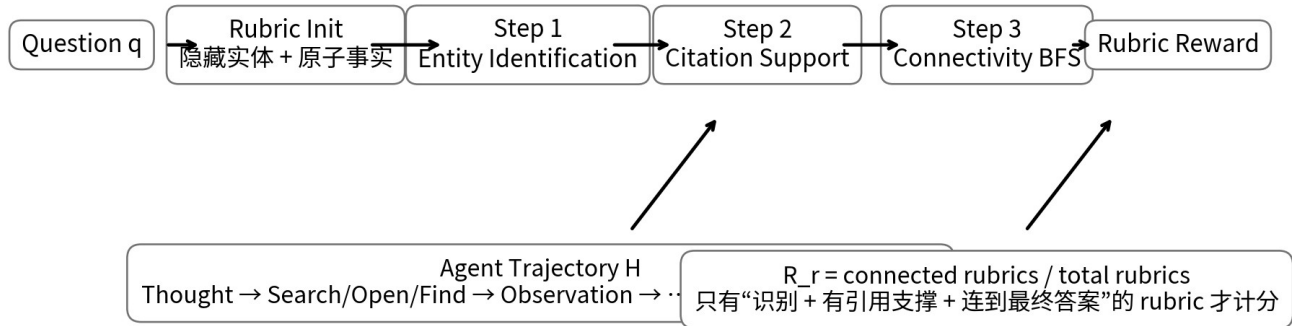


图 3 | CaRR 的三阶段 reward computation

2.1 Rubric Initialization：从隐藏结构提取检查点

对于训练问题 q ，Rubric Generator 先找出应在研究过程中被揭示的 hidden entities E_q ，并把原多跳问题拆成一组 atomic factual statements R_q 。每条 rubric 只描述一个可核验的局部事实，并用 $\langle E_i \rangle$ 占位符表示隐藏实体。最终答案对应 E_0 。Rubrics 在 RL 开始前预生成，训练期间保持固定。[S1]

Know-why 这利用了 synthetic multi-hop data 的“生成结构”：既问题来自一条多跳路径，那么每一跳天然就是 process checkpoint。相比人工写 process labels，成本更低、可扩展。

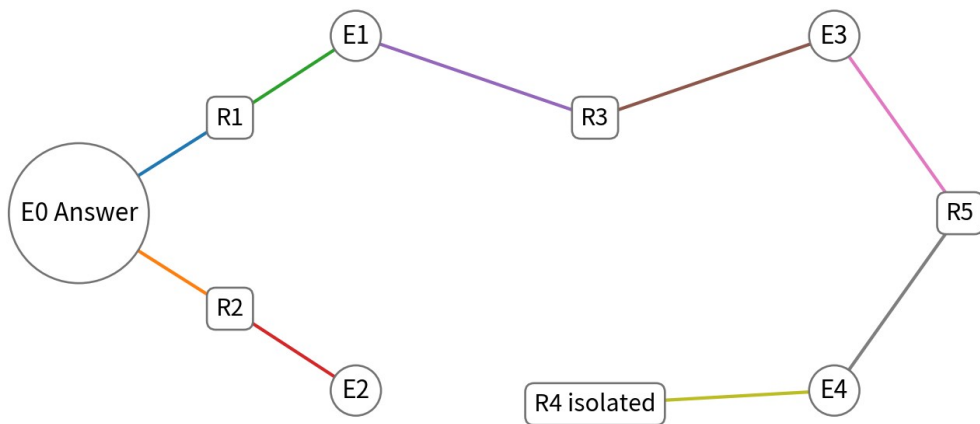
2.2 Step 1：Hidden Entity Identification

Judge LLM 首先只看最终响应，判断每个 hidden entity 是否被显式识别并写出具体名称。只有一条 rubric 涉及的所有隐藏实体都被识别后，才进入下一阶段。这里论文刻意不要求模型识别出的实体必须等于造题时的 gold entity；后续 citation support 会决定这个实例化事实是否真的成立。[S1]

2.3 Step 2：Citation-based Rubric Judgment

CaRR 从最终回答中抽取被引用的 URL，再回到 trajectory 中收集这些 URL 对应的 search snippets、open 内容与 find 匹配片段，形成 supporting context C^H 。Judge LLM 判断每条已实例化 rubric 是否被这些真实网页内容完整支持。论文最多抽取 20 个 cited URLs，明确是为了防止模型通过“疯狂堆引用”进行 reward hacking。[S1]

2.4 Step 3 : Evidence Connectivity Check



BFS 从预测答案 E0 出发：被支持但不在答案证据链上的孤立 rubric 不计分

图 4 | CaRR 把“引用正确”进一步升级成“证据链连通”

仅仅存在若干独立正确事实仍不够。CaRR 构造“实体节点 + 已支持 rubric 节点”的二部图，如果某实体出现在某 rubric 中就连边，然后从预测答案 E0 做 BFS。只有能从 E0 访问到的 supported rubrics 才被计入最终 reward。最终 $R_r = |\text{connected rubrics}| / |\text{all rubrics}|$ 。[S1]

关键洞察 Evidence Connectivity 是 CaRR 和普通 citation reward 的分水岭：它不是问“你有没有引用”，而是问“这些引用能不能共同证明你的答案”。

3. C-GRPO：怎样把 CaRR 接入 RL



$$R = (1-\alpha) \cdot R_o + \alpha \cdot R_o \cdot \hat{R}_r \quad (\text{论文 } \alpha=0.3)$$

Know-how：不要在训练早期给“答错但过程部分正确”的轨迹正优势，否则会把错误最终答案一起强化

图 5 | C-GRPO 的 Correct-only Rubric Bonus

C-GRPO 在标准 GRPO 的 outcome reward 上叠加 CaRR，但它采用一个非常保守的 gate：只有 outcome reward = 1 的正确 rollout 才能获得 rubric bonus。混合 reward 为 $R=(1-\alpha)R_o + \alpha \cdot R_o \cdot \hat{R}_r$ ，其中 $\hat{R}_r$ 会在同一 rollout group 内按最大 rubric reward 归一化。论文主实验 $\alpha=0.3$ 。[S1]

3.1 为什么不是给所有 rollout 都加 process reward？

直觉上，“即便答错，如果中间过程不错也应该奖励”。但论文的消融得到相反结果：训练早期正确 rollout 很少、overlength rollout 很多，此时部分错误轨迹可能凭几个局部 rubric 获得相对较高 reward，进而得到正 advantage，导致 policy 被往“最终仍然答错”的方向优化。给所有 rollout 加 rubric reward 后，4B 模型四个 benchmark 都明显下降。[S1]

3.2 Observation masking / token-level RL

论文的 multi-turn GRPO objective 只对 LLM 自己生成的 token 计算 loss；网页 observation token 通过 indicator $I(H_{i,j})$ 被排除。这是 agentic RL 的基本但重要原则：环境返回内容属于 observation，不应当被当成 policy action 来优化。格式错误或超过 token/tool-call budget 的 rollout 直接 reward=0。

4. 训练数据、工具环境与参数

CaRR 不重新发明数据合成，而是直接复用 DeepDive：由知识图谱随机游走与 entity obfuscation 自动生成高难 multi-hop QA。数据包含 1,016 条 SFT 样本与 2,234 条 RL QA；CaRR 的实验用其中 832 条经过 GLM-4.6 reject sampling 的高质量轨迹做 cold-start SFT。[S1][S3]

层	论文配置	Know-how
Backbone	Qwen3-4B-Thinking-2507; Qwen3-30B-A3B-Thinking-2507	同时验证 dense 与 MoE，不把收益绑定到单一架构

层	论文配置	Know-how
SFT	832 traces; 3 epochs; batch 16; lr 4e-5; max context 128k	先建立基本 tool/citation 语法, 降低 RL 冷启动不稳定
RL	2,234 QA; rollout size 16; 8 samples/prompt; global batch 128; T=1.0; lr 2e-6; 3 epochs; 64k	高样本多样性让 group-relative reward 有足够对比
Rubric	$\alpha=0.3$; DeepSeek-v3.2 judge	Rubric 是辅助目标, 不应压过“答对”主目标
Tools	Serper Search; Jina Open 前 10k chars; Find=字符串匹配	工具能力刻意简单, 实验收益主要来自 reward 设计而非浏览器工程

4.1 当前开源工程结构

截至 2026-08-13, 官方 THU DM/CaRR 仓库为 MIT 许可, README 公告 2026-03-14 已完整开源 RL training 与 evaluation code, 2026-04-08 论文被 ACL 2026 接收。仓库顶层主要包含 data、deepsearch_rm_with_rubrics、scripts、slime、tool_server 等目录。当前 README 把 CaRR reward evaluator 做成远程 reward-model server, 并用 Slime 作为 RL 训练框架。[S2]

工程边界 Paper Method 与 Repo Runtime 要区分: 论文主实验写的是 DeepSeek-v3.2 judge; 当前仓库启动脚本还提供 deepseek-chat 的 training RM server 与 GPT-5-Chat evaluation RM server。这属于复现实用配置, 不应与论文原始实验设置混为一谈。

5. 实验结果：C-GRPO 是否真的更强？

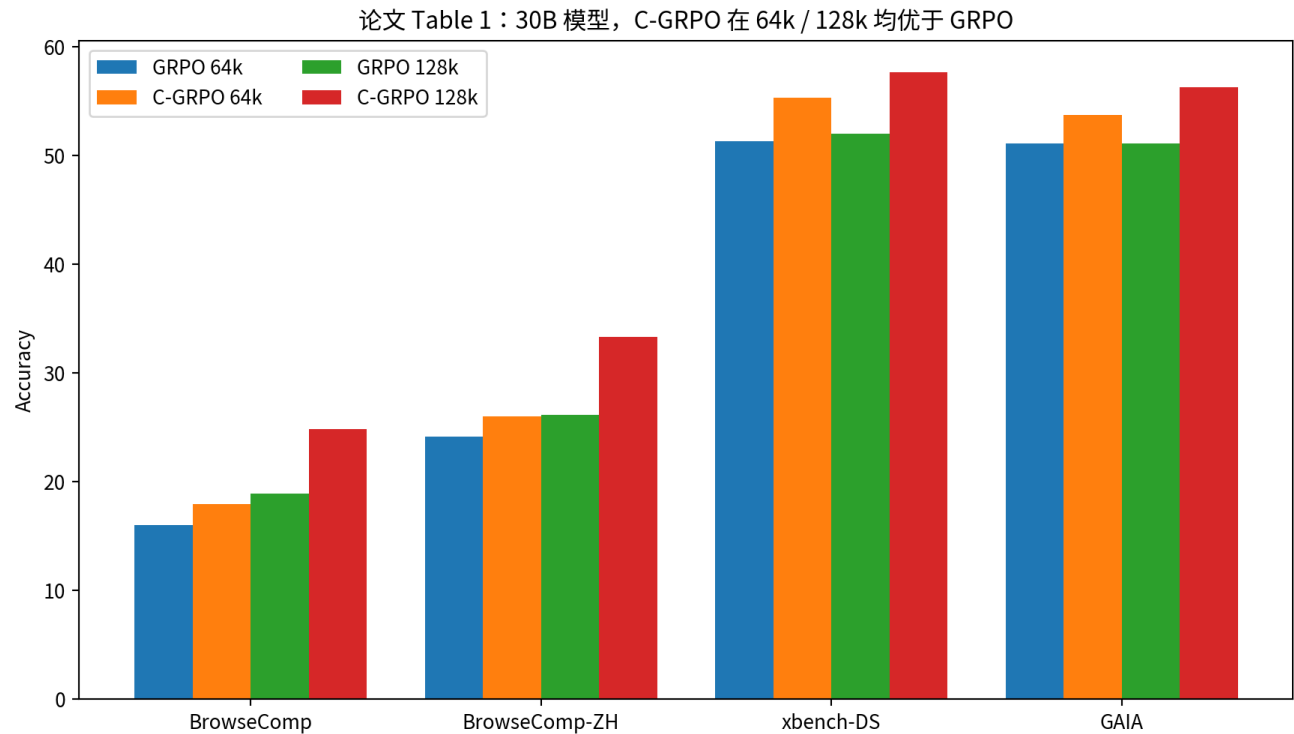


图 6 | 论文 Table 1：30B 主结果

四个 benchmark 上，C-GRPO 在 4B 与 30B、64k 与 128k 的所有设置都优于纯 GRPO 与 E-GRPO。论文汇总：相较 GRPO，4B 在 64k/128k 的平均增益分别为 +5.1/+8.0；30B 为 +2.6/+6.0。尤其值得注意的是：纯 GRPO 虽然在训练长度 64k 内明显改善 SFT，却会削弱 128k 的 test-time scaling；C-GRPO 则在两种 context budget 都持续改善。[S1]

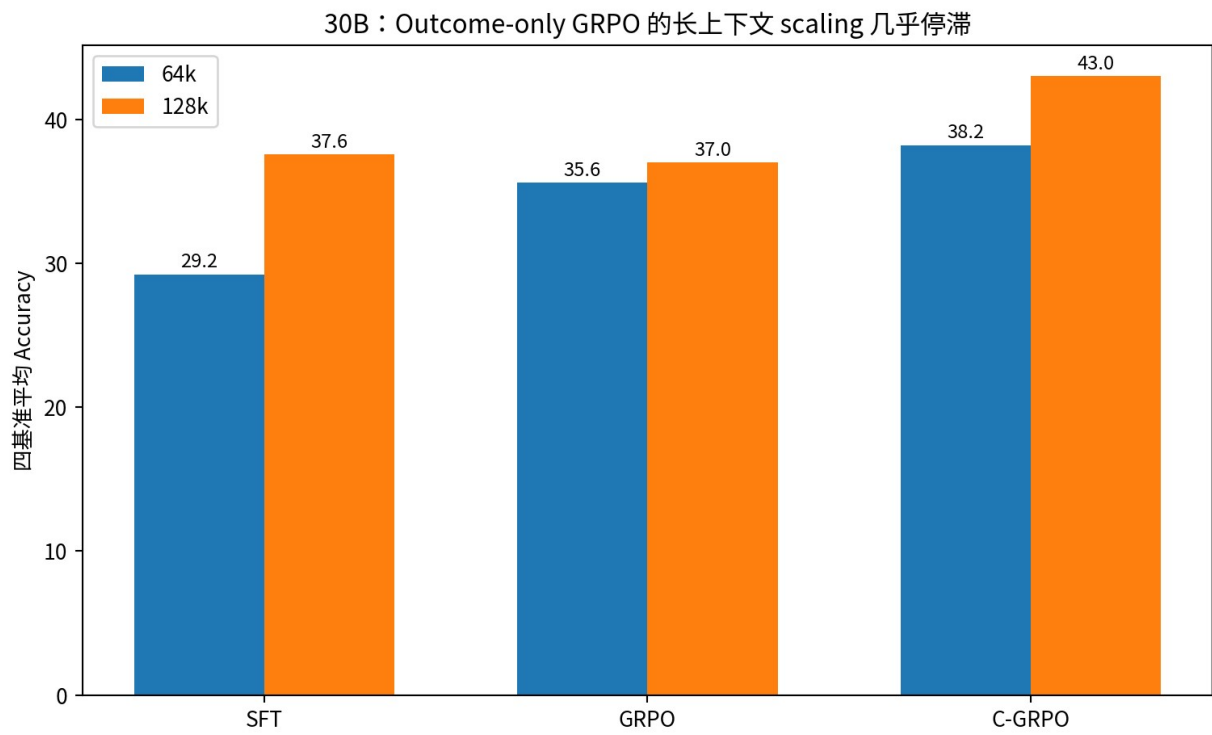


图 7 | 30B 四基准平均：C-GRPO 保留更好的长上下文 Test-time Scaling

Know-why 对 Research Agent 来说，“加预算还能不能继续变强”比单个固定预算分数更能反映 policy 是否真的学会验证。Shortcut policy 往往在短预算看起来高效，但长预算并不会利用额外证据。

6. 训练动态：为什么 C-GRPO 更“像研究”

论文 Figure 4 的最有价值观察不是 reward 曲线本身，而是 tool-call 行为。训练初期，GRPO 与 C-GRPO 都会减少无效搜索以避免 overlength；之后 GRPO 继续压缩工具调用，逐渐偏向 shortcut；C-GRPO 的 tool steps 则重新上升，因为满足更多 rubric 需要额外证据与验证。与此同时 C-GRPO 的 outcome reward 并没有因此变差，反而略高。[S1]

论文 Table 2：C-GRPO 搜集更多证据并满足更多 Rubrics

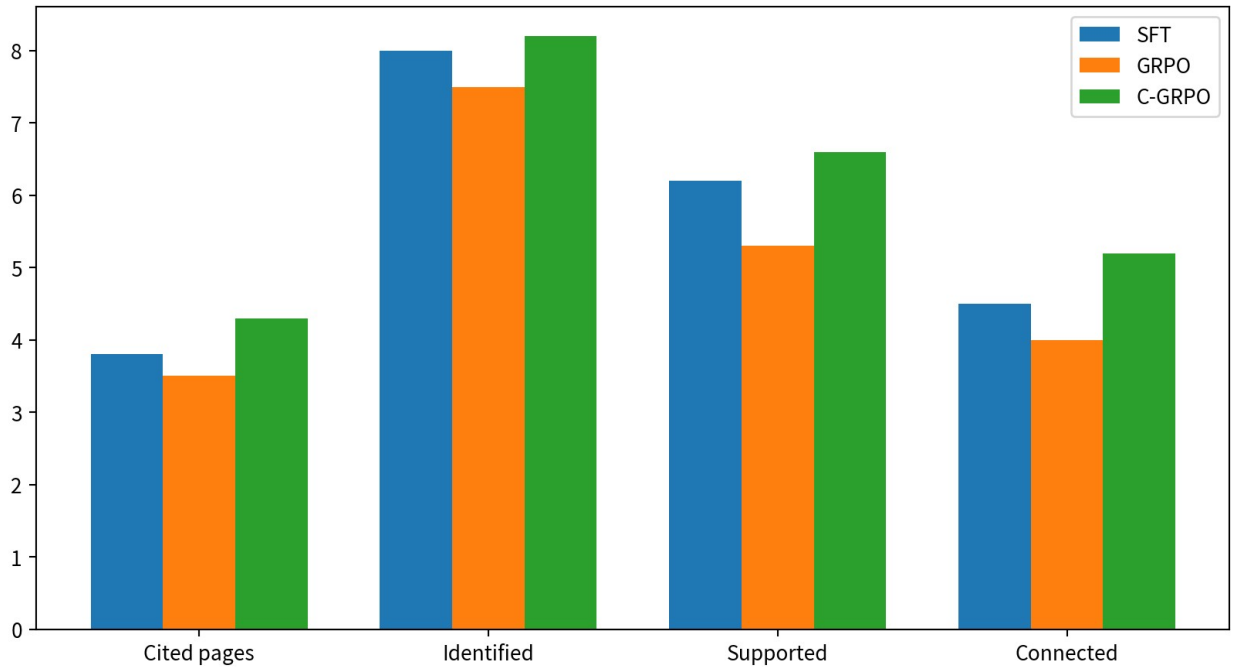


图 8 | 同样答对的 BrowseComp 子集：C-GRPO 引用更多网页、满足更多证据 Rubrics

论文在 30B、且三种模型都答对的 BrowseComp 子集上进一步比较：SFT 平均引用 3.8 个网页、connected rubrics 4.5；GRPO 下降到 3.5 与 4.0；C-GRPO 上升到 4.3 与 5.2。这直接说明 outcome-only RL 可能让模型比 SFT 更“少验证”，而 CaRR 把研究完整性重新纳入优化目标。[S1]

7. Open-ended Deep Research 泛化

虽然 CaRR 的训练题是 short-form synthetic QA，作者仍在 DeepResearch Bench 上测试长篇 PhD-level research report。论文使用当时 benchmark 的 Gemini-2.5-Pro-preview rubric evaluator。30B C-GRPO 总分 41.99，高于同 backbone 的 SFT 37.51、GRPO 39.30、E-GRPO 36.12，也高于论文表中的 Tongyi-DeepResearch 40.46 与 Grok-Deeper-Search 38.22，但仍低于 OpenAI-DeepResearch 46.45 与 Kimi-Researcher 44.64。[S1]

Model	Overall	Comp.	Insight	Instruction	Readability
DeepDive-30B-SFT	37.51	34.27	28.85	46.77	43.21
+ GRPO	39.30	36.10	31.66	47.65	44.92
+ C-GRPO	41.99	39.75	35.87	48.51	46.63
Tongyi-DeepResearch (paper ref.)	40.46	39.46	34.44	46.22	44.27
OpenAI-DeepResearch (paper ref.)	46.45	46.46	43.73	49.39	47.22

评测边界 DeepResearch Bench 在 2026 年更换过官方 evaluator。因此这里应被视为“论文当时协议下的历史结果”，不要与后续 GPT-5.5 evaluator 的新 leaderboard 做直接数值对比。

8. Ablation：真正不可缺的组件是什么？

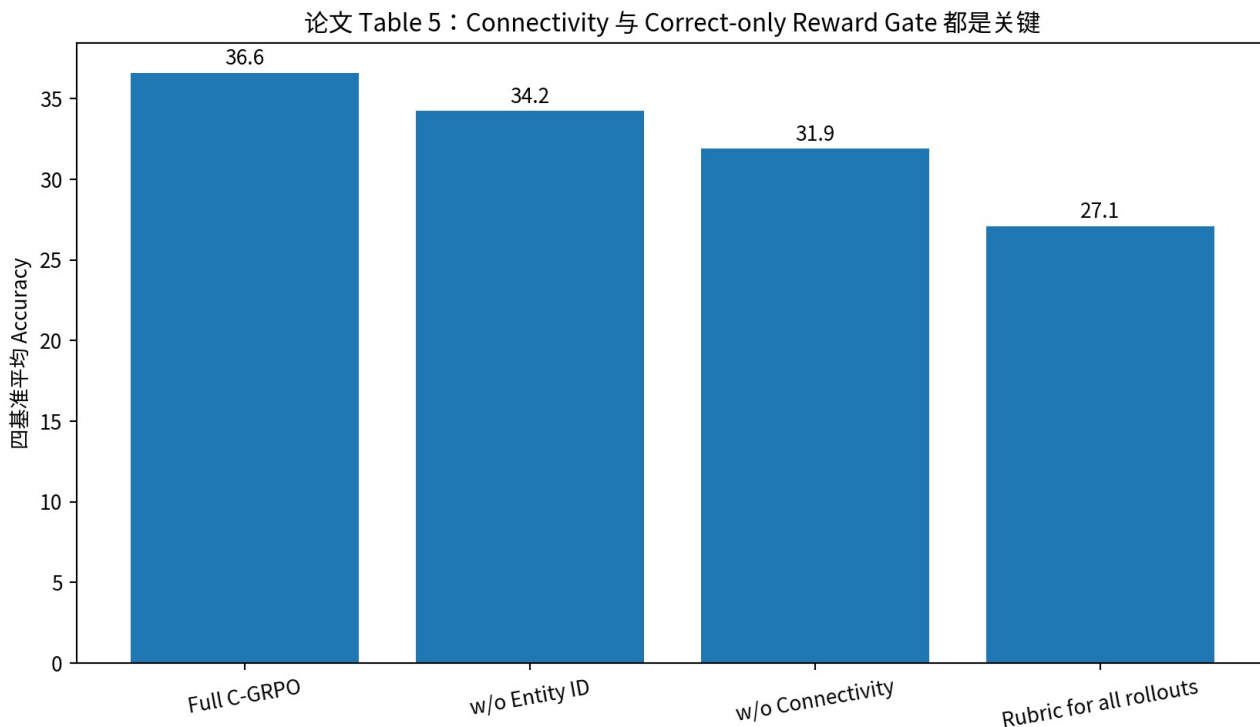


图 9 | 论文 Table 5 的四基准平均

8.1 $\alpha=0.3$ 是“主目标 vs 过程目标”的平衡点

α 从 0（退化为 GRPO）提高到 0.3 时总体性能持续改善；提高到 0.5 后下降。说明 rubric reward 最适合作为“正确答案上的质量排序器”，而不是取代 outcome objective。

8.2 Hidden Entity Identification 提高 Judge 严格性

如果跳过实体识别，直接让 Judge 根据响应与网页判断 rubric 是否被支持，四基准平均会下降。这一步相当于先要求模型把隐含推理变量显式写出来，让后续事实验证有明确对象。

8.3 Connectivity 是防 Reward Hacking 的关键

去掉 connectivity，模型可以找到一组彼此独立但局部正确的事实来“刷 rubric”，即使这些事实与最终答案并无因果/逻辑连接。论文中这是比去掉 entity identification 更严重的性能下降。

8.4 Correct-only Gate 是 RL 稳定器

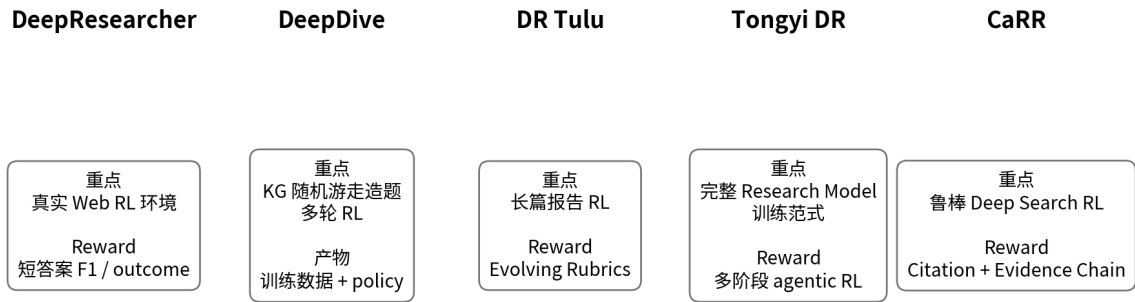
把 rubric reward 给所有 rollout 是最差的消融之一。它提醒一个通用 Agent RL 原则：过程奖励如果不能保证与最终任务成功方向一致，就可能制造错误 credit assignment；尤其是在训练早期 success rate 很低时。

9. CaRR vs DR Tulu：两个 Rubric RL 的根本差异

维度	CaRR	DR Tulu / RLER
主要任务	Deep Search QA，短答案可验证；再测试 open-ended 泛化	直接优化长篇 Deep Research report
Rubric 来源	从 synthetic multi-hop question 的 compositional structure 预生成	Persistent search-grounded + rollout 对比产生 evolving rubrics
训练期是否变化	固定，不随 policy 更新	动态演化，Rubric Buffer 持续更新
Grounding	显式从 cited URLs 回收 trajectory web content	Rubric 可由搜索增强，并有 citation/long-form judge
结构约束	实体-rubric 二部图 + BFS connectivity	主要靠 rubric 集合的判别力/覆盖与 buffer 选择
Reward gate	只对 outcome=1 的 rollout 加 rubric bonus	面向长篇 response 的 rubric score 直接参与 RL
最适合借鉴	鲁棒搜索、证据链、反 shortcut、citation-aware verifier	长篇报告、开放任务、Verifier co-evolution

两者不是互斥路线。一个非常自然的下一代设计是：用 DR Tulu 的 evolving rubrics 负责发现“当前 Policy 还缺哪些质量维度”，再用 CaRR 的 citation grounding + evidence connectivity 把其中事实性 rubric 变成可验证的证据图约束。

10. 与 Deep Research 体系的统一位置



CaRR 的位置：不是另一个搜索 Workflow，而是可插入 Research Agent 训练栈的 Evidence-aware Reward / Verifier Layer

图 10 | CaRR 更像 Research Agent 的 Reward / Verifier Layer

把当前几类系统放到同一条链上，可以看到研究对象不断下沉：GPT Researcher / STORM / Open Deep Research 主要设计 workflow；MindSearch / WebThinker 研究推理时搜索策略；DeepResearcher / DeepDive / Tongyi 研究如何训练 Research Policy；DR Tulu 与 CaRR 则开始研究“怎样构造不会被 Agent 套路化的 verifier”。

核心抽象 Research Agent = Policy × Tool Environment × Evidence State × Verifier × Runtime Guardrails. CaRR 的最大贡献集中在 Verifier：让 reward 从“答案字符串”升级为“答案背后的证据结构”。

11. 可迁移 Know-How：如何在自己的 Research Agent 中复用

1. 先定义 Evidence Contract：最终回答中的关键实体与事实必须显式化，引用必须能回溯到真实 tool observation。
2. 把复杂任务拆成 Atomic Rubrics，但不要只用自然语言 Judge；对可结构化关系增加确定性检查。
3. 建立 Citation Store：每次 Search/Open/Find 返回 source_id/URL/content_hash，最终 citation 直接引用这些 ID。
4. Reward 计算不要重新联网搜证据，而应优先验证“Agent 当时实际看过的 evidence”，否则 verifier 会替 policy 做研究。
5. 为 evidence 构图：Claim/Rubric $\leftrightarrow$ Entity $\leftrightarrow$ Source，做 reachability / coverage / contradiction 检查。
6. 限制引用数量与重复来源，避免 citation spam reward hacking；同时保留 diversity 指标。
7. 正确答案与过程奖励分层：Outcome 负责方向，Process/Provenance 负责在成功轨迹中排序质量。
8. 监控 tool-call curve、evidence coverage、long-budget scaling，而不仅是 train reward 与 final accuracy。
9. 把 Reward Model 做成独立服务：Trainer 只消费结构化 reward breakdown，便于替换 Judge 与离线重放。
10. 上线前做 verifier adversarial eval：孤立事实、伪引用、同 URL 重复引用、答案先猜后补证据、引用不支持 claim 等。

12. 一个生产级 CaRR-style Verifier 的建议架构

建议把论文方法从“训练时 reward”扩展成训练与推理共用的 Evidence Verifier：

- Rubric Generator：输入 Research Brief / synthetic path，输出 atomic claims、hidden variables、dependency edges。
- Provenance Ledger：保存每次工具调用的 source、时间、query、snippet/opened content、hash 与 lineage。
- Entity Resolver：从 final answer / notes 中抽取实体并映射到 rubric placeholders。
- Citation Entailment Judge：只允许用 ledger 中实际见过的 source 判断 claim support。
- Evidence Graph：构建 Claim/Entity/Source 多部图，检查从核心结论到证据的可达性。
- Reward Composer：Outcome、Coverage、Citation Precision、Connectivity、Cost、Overlength 等分项组合。
- Anti-hacking Rules：citation cap、duplicate-source penalty、uncited-claim penalty、unsupported-URL rejection。
- Offline Replay：保存 trajectory + reward breakdown，允许升级 verifier 后重算历史数据而无需重新 rollout。

更进一步 如果任务是开放式报告，可将“固定 Rubrics”升级为 Persistent + Evolving Rubrics；但事实型 rubric 仍用 CaRR 式 citation/evidence graph 验证。这样把 DR Tulu 的探索性与 CaRR 的证据严谨性结合。

13. 局限与需要谨慎解读的地方

- CaRR 的 rubric initialization 依赖 synthetic multi-hop question 的 compositional structure；论文自己承认难以直接迁移到需求隐式、答案开放的训练问题。

- Citation support 仍由 LLM Judge 判断，存在 judge bias / calibration / prompt sensitivity；BFS 只保证结构连通，不保证语义推理真的正确。
- “引用更多网页”不是天然更好；CaRR 的实验里它与更高 rubric satisfaction 同时出现，但生产系统仍需加入来源质量、重复性和成本约束。
- 当前实验主要验证搜索 QA 与 open-ended transfer，而不是直接在大规模开放报告数据上用 CaRR 训练。
- 测试依赖在线搜索环境，搜索结果变化会影响复现；需要 cache、snapshot、source hashing 与可重放环境。

14. 最终结论：Deep Research RL 正从 Reward 走向 Verifier Engineering

结论 CaRR 的意义不是多加一个 reward term，而是把“研究是否可靠”形式化成一个可计算的 Evidence Contract: Find the entities → Ground the facts → Connect the evidence → Then reward the correct answer.

这条路线说明，未来 Research Agent 的核心竞争力不会只来自更长上下文、更大模型或更多搜索次数，而会越来越取决于：系统是否能保存 provenance、能否把 claim 对齐到 evidence、能否发现 shortcut/hallucination、以及 verifier 是否足够难被 policy 钻空子。

可以将这一演进概括为：

统一公式 Robust Deep Research RL = Outcome Success × Evidence Coverage × Citation Grounding × Evidence Connectivity × Stable Credit Assignment

在工程实践中，CaRR 最适合作为一个独立的“Evidence-aware Reward / Verification Layer”，叠加到已有 Research Runtime 上，而不是取代 Planner、Search、Memory 或 Report Writer。

附录 A | 论文关键实验数字

设置	BrowseComp	BrowseComp-ZH	xbench-DS	GAIA
4B GRPO 64k	12.9	16.6	41.0	40.5
4B C-GRPO 64k	13.9	18.2	50.3	48.9
4B GRPO 128k	14.7	17.5	41.3	41.1
4B C-GRPO 128k	17.5	24.7	54.0	50.2
30B GRPO 64k	16.0	24.1	51.3	51.1
30B C-GRPO 64k	17.9	26.0	55.3	53.7
30B GRPO 128k	18.9	26.1	52.0	51.1
30B C-GRPO 128k	24.8	33.3	57.7	56.3

附录 B | 源码阅读地图

路径/模块	作用	阅读重点
README.md	项目入口与复现顺序	论文/代码开源时间、Tool/RM/Training/Eval 启动链
deepsearch_rm_with_rubrics/	CaRR reward server	Rubric generation/judgment、citation-aware reward 的服务化边界

路径/模块	作用	阅读重点
tool_server/	搜索与网页工具服务	Serper/Jina、session、tool observation 的运行时接口
scripts/setup/	环境与服务启动	prepare_env、tool server、training/eval RM、数据解密
scripts/training/	4B / 30B C-GRPO 训练脚本	α 、rollout、context、Slime 参数与资源拓扑
scripts/eval/	复现实验	evaluation RM、benchmark 数据与 context/tool budget
slime/	底层 RL 框架	rollout / trainer / distributed runtime; CaRR 在其上集成

附录 C | 主要资料来源与证据边界

- [S1] Zhang et al., “Chaining the Evidence: Robust Reinforcement Learning for Deep Search Agents with Citation-Aware Rubric Rewards” , arXiv:2601.06021 (2026). 本报告方法、公式、实验、消融和局限的主要来源。
- [S2] THU DM/CaRR 官方 GitHub README 与当前仓库结构（访问快照 2026-08-13）。用于代码开源状态、目录结构、Slime / tool server / reward server 与复现步骤。
- [S3] THU DM/DeepDive 官方项目与论文。用于说明 CaRR 所复用的 KG random walk + entity obfuscation 数据来源与上游训练范式。
- [S4] DeepResearch Bench 官方项目。用于理解开放式报告评测；注意该 benchmark 在 2026 年更新过 evaluator，因此 CaRR 论文中的 Gemini-2.5-Pro-preview 分数属于历史协议。

证据分层说明：凡涉及论文方法与数值，均以 [S1] 为准；凡涉及 2026-08-13 当前仓库组织/开源状态，以 [S2] 为准；跨系统比较与“下一代架构”部分属于基于这些一手材料的工程综合与推论，已用“建议/可以理解为/下一代”措辞与论文原结论区分。